NOTEBOOK 00

Python for Actuaries

Seminar Agenda

- Day 1 · Tuesday, 3 November (online)
  - 01a Python Basics: numbers, truth values, if/else, loops
  - 01b Functions: def, lambda, import
  - 02a Objects: classes, inheritance, special methods
  - 02b Data Structures: strings, lists, dicts, sets
  - 02c Files: reading and writing files
- Day 2 · Thursday, 5 November (Ljubljana)
  - 03 NumPy and Pandas: arrays, DataFrames, groupby
  - 04 Data Visualization: Matplotlib, Seaborn, correlations
  - 05 Data Preparation: cleaning, features, train/test split
  - 06 Data Exploration: hands-on exercise
- Day 3 · Friday, 6 November (Ljubljana)
  - 07 Machine Learning: fit, predict, four models
  - 08 GLMs for the average claim amount, or a session on LLMs and Generative AI
  - 09 Fraud Detection: final exercise

Instructors

Prof. Dr. Alexander Brandt

Email: alexander.brandt@dhbw-stuttgart.de

Prof. Dr. Jonas Offtermatt

Email: jonas.offtermatt@dhbw-stuttgart.de

Literature

- The Python Tutorial docs.python.org
- Eric Matthes, Python Crash Course, No Starch Press
- Wes McKinney, Python for Data Analysis, O'Reilly

Typical Unit

Some of the lectures will be provided and conducted in the form of Jupyter Notebooks (jupyter.org). The goal is to program together, and there will also be exercises for each unit that you will need to work on during the lecture.

Why Python?

Python ...

- is *interpreted*
- is *dynamically typed* (as opposed to statically typed)
- is *automatically memory-managed*
- supports *procedural*, *object-oriented*, *imperative*, and *functional* programming paradigms
- was (mostly) developed by a single person: Guido van Rossum (also known as the "benevolent dictator"), and therefore has a quite *opinionated* design
- has a single reference implementation (CPython)
- Version 3 (the latest version) is *not backward compatible* with Version 2, although the latter is still widely used
- has an interesting programming philosophy: "There should be one — and preferably only one — obvious way to do it." (a.k.a. the "Pythonic" way) — see The Zen of Python

In case you are still wondering why:

https://xkcd.com/353/